

ReAct-like工具使用循环

🔗 chatgpt.com/share/6a536bf0-8794-83ec-ba61-65c3fd584e57

You said:

不过 Claude 的原始 reasoning 通常是 protected thinking，因此源码里主要能观察到的是：Action → Observation → Action → Observation 所以叫 ReAct-like tool-use loop 比“严格 textbook ReAct”更准确。怎么理解

ChatGPT said:

可以把这句话理解成：我们能看到 Claude 像 ReAct 一样工作，但看不到它是否真的按照经典 ReAct 的完整格式思考。

经典的 textbook ReAct 通常写成：

Thought：我需要先查天气

Action：调用天气工具

Observation：北京今天 30°C，有雷阵雨

Thought：用户还问是否需要带伞

Action：根据天气结果继续判断

Observation：有降雨概率

Final Answer：建议带伞

它的核心循环是：

Thought → Action → Observation → Thought → Action → Observation

其中：

- **Thought**：模型显式写出的推理过程
- **Action**：模型发起工具调用
- **Observation**：工具返回的结果

但 Claude 的 thinking/reasoning 往往是受保护的，也就是内部推理不会原样暴露给应用代码。开发者实际能看到的通常更像：

Action：搜索相关资料

Observation：搜索结果

Action：打开某个页面

Observation：页面内容

Action：调用计算器

Observation : 计算结果

Final Answer :

所以外部日志表现为：

Action → Observation → Action → Observation

这并不意味着中间没有推理。更准确地说，推理可能发生在每次工具调用之前和之后，只是被系统隐藏了：

[隐藏推理] → Action → Observation → [隐藏推理] → Action → Observation

因此称它为 **ReAct-like tool-use loop**，是在说它的可观察行为模式与 ReAct 相似：

1. 模型根据当前上下文决定下一步行动；
2. 调用工具；
3. 读取工具结果；
4. 再决定下一步行动；
5. 最后生成回答。

但不称为“严格 textbook ReAct”，是因为我们无法从外部确认以下事项：

- 内部是否真的生成了一个显式的自然语言 Thought
- 推理是否严格按照 Thought → Action → Observation 的格式交替
- 内部可能使用连续隐藏状态、压缩推理或其他规划机制，而不是论文中那种可打印的文本轨迹
- 多次工具调用也可能由不同的调度机制实现，不一定是原始 ReAct 算法

可以用一个类比理解：

看到一个人不断“看地图 → 转弯 → 看路牌 → 再转弯”，可以说他的行为像导航算法；但看不到他脑中的路线规划，就不能断言他严格执行了某个特定导航算法的伪代码。

所以这句话强调的是一个重要区分：

行为层面：像 ReAct

机制层面：无法确认是严格 ReAct

“ReAct-like”不是说 Claude 少了推理，而是说：推理被隐藏后，我们只能根据工具调用轨迹描述其外部控制循环，不能把不可见的内部机制直接等同于经典 ReAct。

You said:

那么经典的react模型是否需要将上一轮的thought带到这一轮的prompt中呢

ChatGPT said:

需要。在经典 ReAct 中，上一轮的 Thought 会与 Action、Observation 一起，作为当前任务的历史轨迹 / scratchpad 进入下一轮模型上下文。

典型过程是：

第 1 轮输入：

Question: 北京明天适合爬山吗？

模型输出：

Thought 1: 我需要先查询北京明天的天气。

Action 1: search_weather("北京明天")

工具执行后，Agent 控制器将结果追加到轨迹：

Question: 北京明天适合爬山吗？

Thought 1: 我需要先查询北京明天的天气。

Action 1: search_weather("北京明天")

Observation 1: 明天有中雨，最高温度 28℃。

然后把整个轨迹再次交给模型，模型继续生成：

Thought 2: 有中雨，户外爬山可能存在湿滑风险，需要给出不建议爬山的结论。

Final Answer: 明天不太适合爬山.....

因此完整循环实际上是：

Prompt + Question

↓

Thought 1

Action 1

↓ 工具执行

Observation 1

↓

重新调用 LLM，携带：

Question

Thought 1

Action 1

Observation 1

↓

Thought 2

Action 2 / Final Answer

ReAct 论文强调，reasoning trace 的作用就是帮助模型维护、跟踪和更新行动计划；新的 Observation 加入后，模型会基于之前的推理轨迹继续决策。[arXiv+1](#)

但“带到 prompt”容易产生一个误解

不是把上一轮 Thought 塞进固定的 system prompt :

System Prompt:
你是一个 ReAct Agent.....
上一轮 Thought 是.....

而是把它保留在动态上下文 / agent scratchpad里 :

固定部分 :
System Prompt
Few-shot ReAct examples
工具说明

动态部分 :
用户问题
Thought 1
Action 1
Observation 1
Thought 2
Action 2
Observation 2
...

代码上通常类似 :

```
PythonRunscratchpad = ""

while True:
    prompt = system_prompt + question + scratchpad

    output = llm(prompt)

    thought, action = parse(output)

    observation = execute(action)

    scratchpad += f"""
Thought: {thought}
Action: {action}
Observation: {observation}
"""
```

为什么必须保留 Thought ?

假设第一轮是 :

Thought 1:
用户问的是苹果公司创始人出生地 ,
我需要先查清楚创始人是谁。

Action 1:
search("苹果公司创始人")

Observation 只返回：

Steve Jobs、Steve Wozniak、Ronald Wayne

如果下一轮只传 Observation：

Observation:
Steve Jobs、Steve Wozniak、Ronald Wayne

模型可能不知道：

- 为什么查这三个人；
- 原始问题是什么；
- 下一步应该查出生地还是公司历史；
- 是否要查所有创始人还是其中一人。

保留 Thought 后，模型能接着原来的计划继续执行：

Thought 2:
苹果有三位联合创始人，问题使用了单数“创始人”，
需要判断用户通常指的是 Steve Jobs，或者分别列出三人。

一个重要例外

如果整个过程是模型一次连续生成：

Thought → Action → Observation → Thought

那么前面的 Thought 天然已经位于模型的 token 前缀中，不需要工程代码“重新添加”。

但是实际工具调用通常会中断生成：

LLM 生成 Action
↓
程序暂停模型
↓
执行工具
↓
重新请求 LLM

这种情况下，Agent 框架必须在下一次请求中重新携带历史轨迹，否则模型会失去前面的 ReAct 状态。

所以可以这样记：

经典 **ReAct** 每一轮都要让模型看到此前的 **Thought**、**Action** 和 **Observation**；不一定叫重新拼接 **prompt**，但必须存在于下一轮上下文中。

如果只保留：

Action → Observation → Action → Observation

而不保留显式 **Thought**，那么它更接近 **ReAct-like 工具循环**，不是严格的 textbook **ReAct**。

You said:

所以claudecode将它自己加密的thinking带到下一轮的上下文中

ChatGPT said:

对，但要说得更准确一点：

不是 Claude Code 自己把明文 **Thought** 加密，而是 **Claude 模型/API** 返回一个不可读但带签名的 **thinking block**，**Claude Code** 在下一次工具调用请求中把它原样带回去。

流程大致是：

第一次请求

用户问题

↓

Claude 返回：

```
[
  thinking: {
    可选的思考摘要,
    signature: "加密/签名后的完整 reasoning"
  },
  tool_use: Read(...)
]
```

Claude Code 执行工具后，构造下一次请求：

```
messages = [
  user: 用户问题,

  assistant: [
    原封不动的 thinking block,
    tool_use: Read(...)
  ],

  user: [
    tool_result: 文件内容
```

```
]
]
```

然后 Claude 继续推理：

Claude :
[新的 thinking]
→ [下一个 tool_use 或最终答案]

Anthropic 官方明确要求：工具调用过程中，上一轮 assistant 返回的 thinking block 必须**完整、未经修改地传回 API**，否则可能破坏推理连续性；API 还会通过密码学签名验证其真实性，修改后会直接报错。[Claude Platform Docs+1](#)

因此，从 Claude Code 源码视角看，可能只是类似：

```
opaque_thinking_block
tool_use
tool_result
opaque_thinking_block
tool_use
tool_result
```

Claude Code 不需要知道 reasoning 具体是什么，只需要承担：

接收 thinking block
→ 保存
→ 原样回传

“下一轮”具体指什么

这里最重要的区别是，工具调用的下一次 API 请求和用户的下一轮对话不是一回事。

工具循环内：必须带

例如：

用户：修复这个 Bug

Claude: thinking₁ + Read 工具调用
工具结果：代码内容

Claude: thinking₂ + Edit 工具调用
工具结果：修改成功

Claude: thinking₃ + Test 工具调用
工具结果：测试通过

Claude：最终回答

虽然底层发生了多次 API 请求，但 Anthropic 把它们概念上视为**同一个 assistant turn 的连续生成过程**。所以之前的 thinking 必须带回来，Claude 才能从暂停的位置继续推理。

可以理解成：

```
[隐藏 Thought1]  
  ↓  
Action1  
  ↓  
Observation1  
  ↓  
[通过加密块恢复 Thought1 的推理状态]  
  ↓  
[隐藏 Thought2]  
  ↓  
Action2
```

新的用户对话轮次：不一定必须带

当 Claude 已经输出最终答案，用户又提出新问题时，是否继续保留旧 thinking，取决于模型和上下文管理策略。部分较新的 Claude 模型会默认保留之前的 thinking block，部分早期模型则会由 API 自动剥离；客户端通常可以继续原样传递，由 API 判断哪些块真正提供给模型。[Claude Platform Docs](#)

所以你的理解可以修正成一句更严谨的话：

Claude Code 会把 API 返回的受保护 thinking block 原样带入工具循环中的下一次请求，从而保持 ReAct 式推理连续性；但 Claude Code 本身既看不到，也不负责理解这段原始 reasoning。

因此，外部虽然只观察到：

Action → Observation → Action

内部更可能是：

```
Protected Thinking1  
→ Action  
→ Observation  
→ 恢复 Protected Thinking1  
→ Protected Thinking2  
→ Action
```

这也是为什么它从行为上非常接近经典 ReAct，只不过 textbook ReAct 使用可见的明文 Thought，而 Claude 使用受保护、原样回传的 thinking block。

ChatGPT can make mistakes. Check important info.